P. KARTIKEYA RAJ

2023001492

AI LAB EXPIERMENT NO 5 :

1.Write a program to solve 8-tiles puzzle problem (Using heuristics)

Problem statement :

It aims to rearrange tiles from an initial configuration to a goal configuration by sliding one tile at a time into the blank space, while minimizing the number of moves using heuristic guidance.

The problem is defined by the following elements:

1. Initial State
A given starting arrangement of the 8 tiles and the blank space.

2. Goal State
A predefined target arrangement of the tiles.

3. State
Any valid configuration of the 3×3 board.

4. Operators (Actions)
The blank tile can move:

- Up
- Down
- Left
- Right
  (if the move is within board limits)

5. State Space
All possible reachable configurations of the puzzle.

6. Path Cost (g)
Number of moves made from the initial state.

7. Heuristic Function (h)
An estimated cost to reach the goal from the current state
(Manhattan Distance is commonly used).

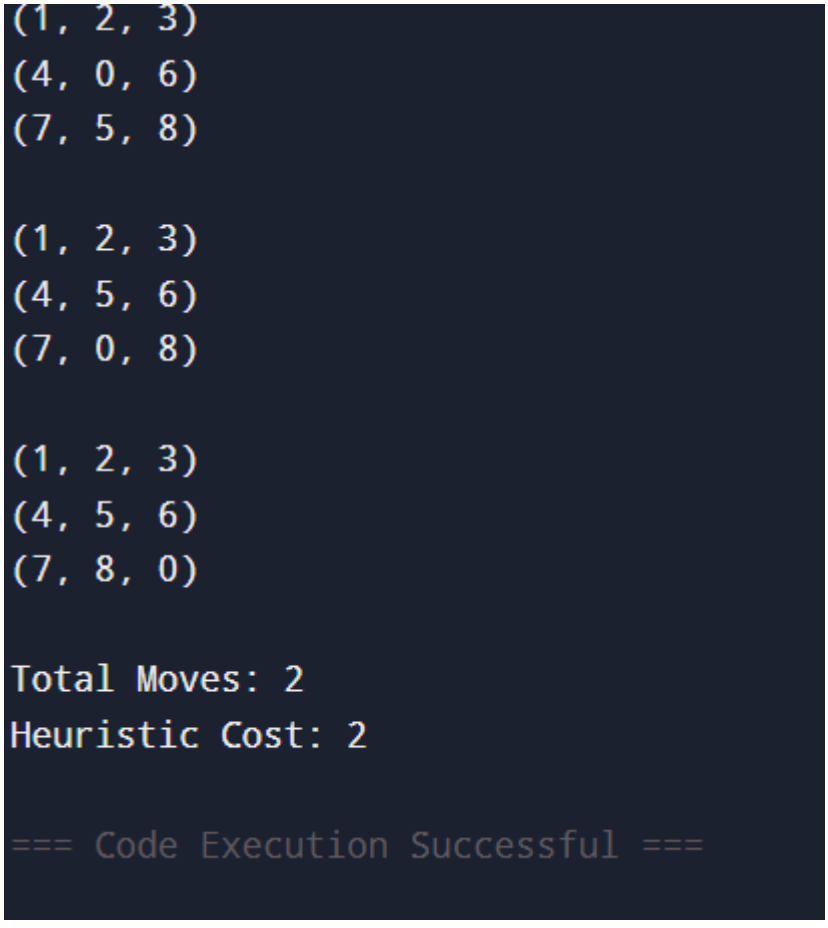
CODE :

```
import heapq


def manhattan(s, g):
    pos = {g[i][j]:(i,j) for i in range(3) for j in range(3)}
```

```
    return sum(abs(i-pos[s[i][j]][0]) + abs(j-pos[s[i][j]][1])
               for i in range(3) for j in range(3) if s[i][j] != 0)

def neighbors(s):
    x,y = [(i,j) for i in range(3) for j in range(3) if s[i][j]==0][0]
    d = [(1,0),(-1,0),(0,1),(0,-1)]
    res=[]
    for dx,dy in d:
        nx,ny=x+dx,y+dy
        if 0<=nx<3 and 0<=ny<3:
            t=[list(r) for r in s]
            t[x][y],t[nx][ny]=t[nx][ny],t[x][y]
            res.append(tuple(map(tuple,t)))
    return res

def astar(start, goal):
    pq=[(manhattan(start,goal),0,start,[])]
    vis=set()
    while pq:
        f,g,s,path=heapq.heappop(pq)
        if s==goal: return path+[s]
        if s in vis: continue
        vis.add(s)
        for n in neighbors(s):
            heapq.heappush(pq,(g+1+manhattan(n,goal),g+1,n,path+[s]))

# Input
start=((1,2,3),(4,0,6),(7,5,8))
goal=((1,2,3),(4,5,6),(7,8,0))

sol=astar(start,goal)
```

```
# Output
for step in sol:
    for r in step: print(r)
    print()
print("Total Moves:",len(sol)-1)
print("Heuristic Cost:",manhattan(start,goal))
```

OUTPUT :



2.Write a program to solve 8-tiles puzzle problem (Using heuristics).

(I) BFS

PROBLEM :

The goal is to find the best path from a source node to a destination/goal node.

- BFS explores nodes **level by level**
- First time reaching destination = **shortest path**
- Path length = number of edges

CODE :

```
def bfs(graph, start, goal):
    q = deque([[start]])
    visited = set()

    while q:
        path = q.popleft()
        node = path[-1]

        if node == goal:
            return path, len(path)-1

        if node not in visited:
            visited.add(node)
            for n in graph[node]:
                q.append(path + [n])

# Input
graph = {
    'A':['B','C'],
    'B':['D'],
    'C':['D'],
    'D':['E'],
    'E':[]
}
```

```
path, length = bfs(graph, 'A', 'E')
print("Shortest Path:", path)
print("Path Length:", length)
```

OUTPUT :

```
Shortest Path: ['A', 'B', 'D', 'E']
Path Length: 3

=== Code Execution Successful ===
```

ii. Lowest-cost :

PROBLEM :

- **Used for weighted graphs**
- **Always expands the path with lowest total cost**
- **Guarantees optimal path**

CODE:

```
import heapq

def lowest_cost(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost

        if node not in visited:
```

```
            visited.add(node)
            for n, c in graph[node]:
                heapq.heappush(pq, (cost + c, n, path + [n]))

# Input
graph = {
    'A':[('B',1),('C',4)],
    'B':[('D',2)],
    'C':[('D',1)],
    'D':[('E',3)],
    'E':[]
}

path, cost = lowest_cost(graph, 'A', 'E')
print("Optimal Path:", path)
print("Total Cost:", cost)
```

OUTPUT: